UNIVERSIDAD RAFAEL NÚÑEZ

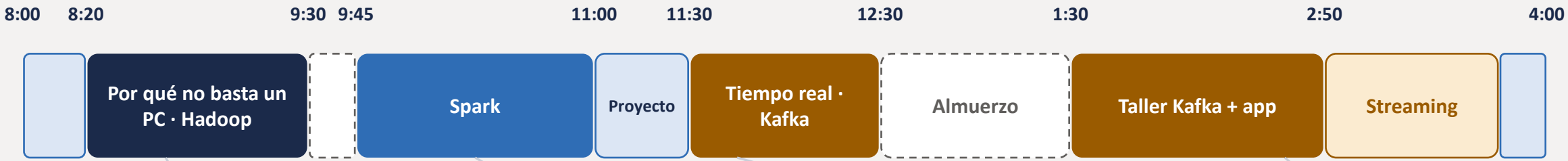
Especialización en Analítica de Datos · Tecnologías de Procesamiento de Datos

El caso por lotes y en flujo

Encuentro 1, Día 2 · Sábado 31 de octubre · 8:00 a. m.–4:00 p. m. · en GitHub Codespaces

Agenda de hoy

La mañana, por lotes; la tarde, en flujo. El mismo caso, todo en tu codespace



8:00–9:30 · Codespace y Hadoop
Preparación Previa todos juntos; Excel y pandas; MapReduce con 31 bloques y el combiner

9:45–11:00 · Spark
El mismo cálculo con CSV y con Parquet, y la Spark UI

11:00–12:30 · Proyecto y tiempo real
Definición del Proyecto Final; del lote al flujo; Kafka

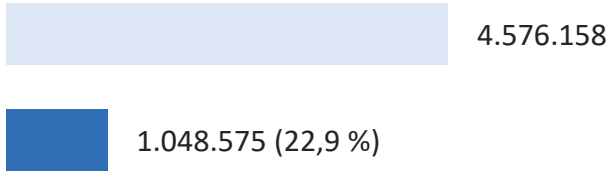
1:30–3:45 · El flujo en vivo
La app de viajes sobre Kafka y Spark Streaming

Un problema por bloque y su solución con evidencia: todo lo de hoy corre en su codespace; el 6 y el 7 de noviembre, el mismo caso en la nube.

¿Por qué no basta un computador?

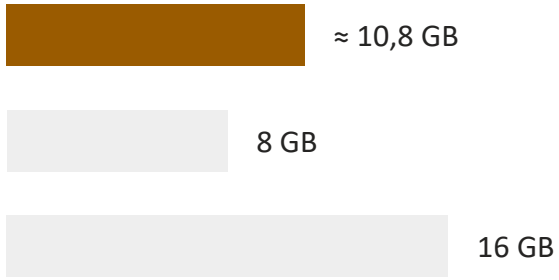
8:20–8:40 · Taller de la mañana, Parte A: el mes de junio de 2025 frente a Excel y pandas

Excel: el mes (4 GB) no abre; la semana no cabe



Llega hasta el 3 de junio a las 21:04: US$ 28,1 millones de tarifa en lugar de los US$ 132,1 millones de la semana.

pandas: memoria para el mes completo



Medido en el codespace: 0,54 GB por millón de viajes. El sistema ya usa parte de la RAM.

- **Límite de la herramienta:** Excel admite 1.048.576 filas por hoja y no avisa en las celdas que faltan datos.
- **Límite de la máquina:** la memoria de un solo equipo; con un año de viajes serían doce veces más.

Escenario: la gerencia pide el informe del mes

Cuántos viajes, cuánto se cobró, cuánto se pagó a los conductores y qué parte quedó en la plataforma.

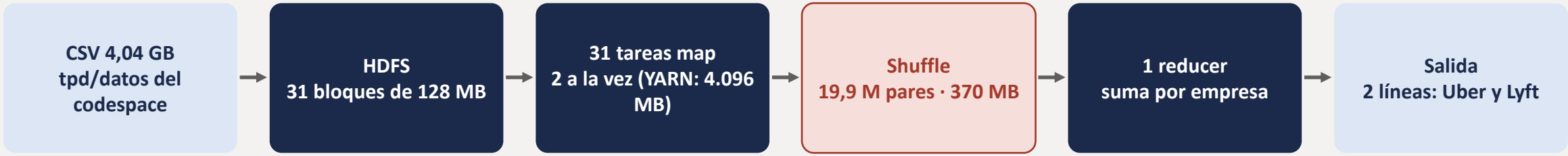
Con un solo computador
Un informe incompleto que parece correcto, o un proceso que no termina.

Repartiendo el trabajo
El archivo en bloques y el cálculo en varias tareas: lo que hace Hadoop desde las 8:40.

El problema no es la herramienta: el dato creció más que la máquina.

Hadoop con el mes completo: del archivo a dos líneas

8:40–9:30 · Taller de la mañana, Parte B: lo que deben encontrar en las consolas



Cómo leer los contadores del job

- **31 splits = 31 bloques = 31 maps:** el paralelismo lo decide el tamaño del archivo.
- **Map input 19.868.010:** los viajes más el encabezado.
- **Spilled 39.736.018:** cada par se escribió dos veces al disco del nodo.
- **Elapsed 3 min 33 s:** los maps tardan 6 s, pero van por tandas.

Optimización: el mismo job con un combiner

El reducer suma dentro de cada map antes del shuffle (sumar es asociativo): cada map envía 2 líneas, 31 × 2 = 62 registros. Resultado idéntico.

Sin combiner
Shuffle 369.900.781 bytes · 213 s

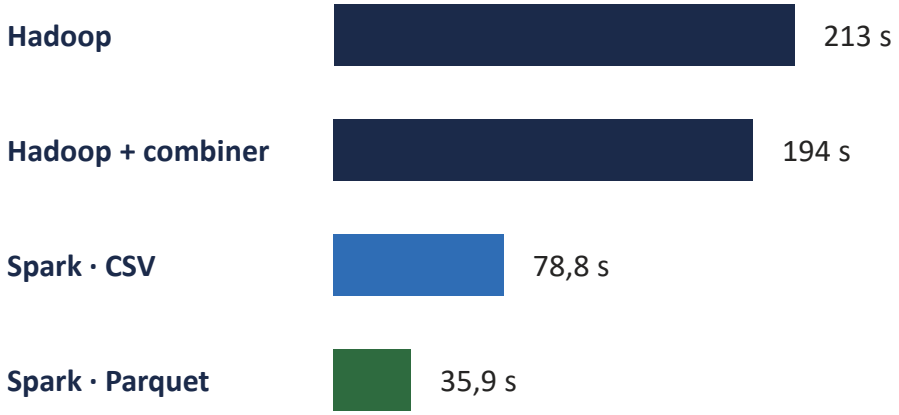
Con combiner
Shuffle 2.344 bytes · 194 s

Optimizar es mover menos datos: el tiempo baja poco aquí, pero en un clúster real el shuffle cruza la red.

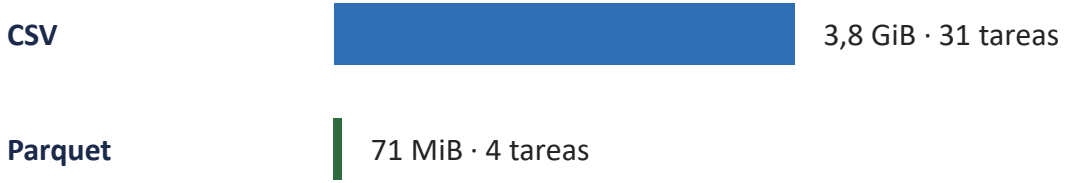
Spark frente a MapReduce: el mismo resultado, otro camino

9:45–11:00 · Taller de la mañana, Parte C: tiempos y datos leídos, medidos en el mismo codespace

Tiempo del mismo cálculo (segundos)



Datos leídos por pasada



- **Agregación parcial:** Spark suma dentro de cada tarea antes del shuffle (7 KB), como un combiner automático.
- **Formato:** Parquet guarda por columnas y comprimido: Spark leyó solo las 3 columnas del cálculo.

Escenario: el informe debe salir cada noche

Mismo resultado al centavo en los cuatro casos; cambia cuánto se lee, cuánto se mueve y cuánto se tarda.

Honestidad de la comparación
Spark usó los 4 núcleos sin el límite de memoria de YARN y leyó del disco del codespace, no de HDFS: no es una carrera igual.

Lo que sí se concluye
Menos disco entre fases y menos datos leídos: ahí está la ganancia.

Elegir motor y formato es una decisión de rendimiento y de costo, con evidencia de la Spark UI.

Cierre de la Unidad 1: el Proyecto Final

11:00–11:30 · Se define con la evidencia de la mañana, no como una tarea aparte

1

La necesidad

Qué decisión del caso de viajes resuelve su arquitectura y para quién (gerencia, operación, regulador).

2

La arquitectura

Hadoop, Spark o híbrida, en el codespace y en la nube; qué formato de datos y por qué.

3

La justificación

Tiempos, contadores y bytes medidos hoy; qué cambia con un año de datos.

4

La sustentación

Sábado 7 de noviembre, 2:15–3:15: configuración, integración de datos y rendimiento (rúbrica).

Pregunta a cada equipo antes de cerrar: ¿en qué escenario del caso elegirían Hadoop puro sobre Spark? Un caso concreto, no una respuesta genérica.

Unidad 3

Procesamiento de datos en tiempo real

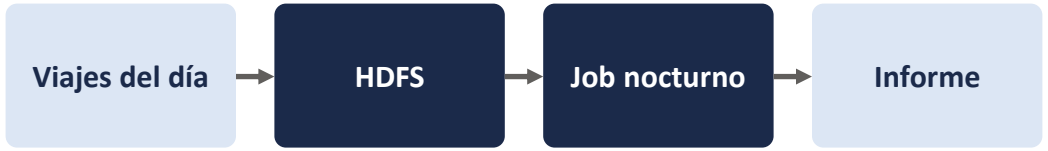
11:30 a. m.–12:30 p. m. · Conceptos y Apache Kafka

> *¿Qué decisión del caso no puede esperar al proceso de las 2 de la mañana?*

Del lote al flujo: la misma cadena, otra urgencia

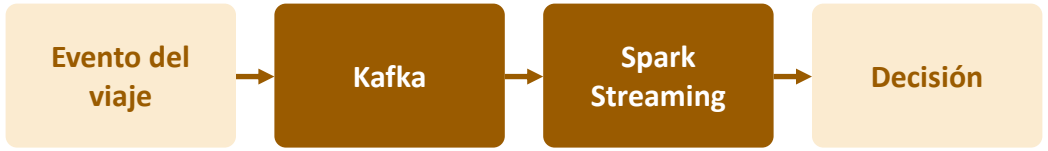
Lo que hicieron en la mañana responde mañana; lo de la tarde responde en segundos

Por lotes (la mañana)



Datos ya acumulados · responde en horas · la pregunta: ¿qué pasó?

En flujo (la tarde)



Cada evento a medida que llega · responde en segundos · la pregunta: ¿qué está pasando ahora? (en el taller: latencia de 2 a 4 ms en Kafka)

- **Lote:** se ejecuta cada cierto tiempo sobre datos acumulados; barato y completo, pero tarde.
- **Flujo:** procesa cada evento al llegar; exige que ningún evento se pierda y que el sistema no se detenga.

Escenario: 7:58 p. m., llueve en Midtown

Suben las solicitudes y bajan los carros libres. El informe de mañana lo confirmará; la tarifa dinámica debe reaccionar ahora.

Si solo hay lotes
La empresa se entera mañana de que perdió viajes esta noche.

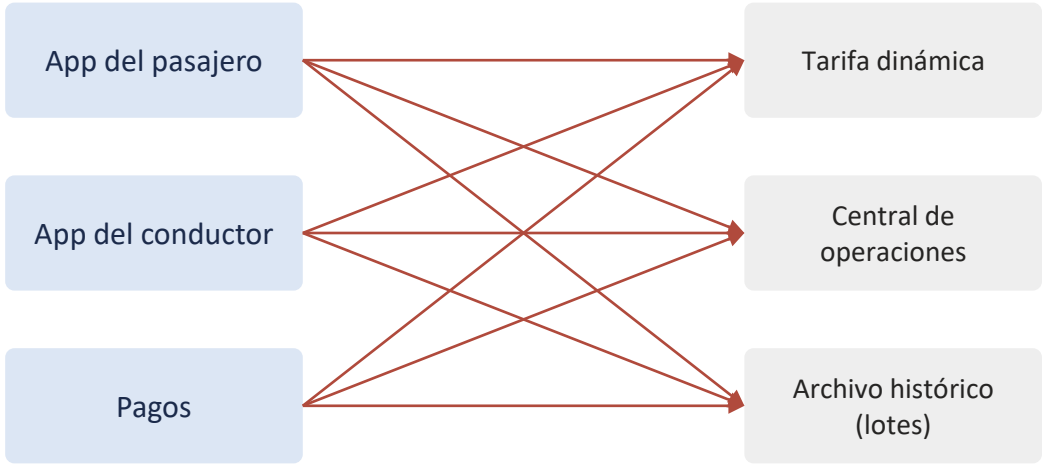
Con flujo y lotes
El flujo decide ahora; el lote consolida y audita después. Los dos leen los mismos eventos.

Kafka es el punto de encuentro: publica cada evento una vez y lo entrega al flujo y al lote.

Por qué Kafka: el modelo publicación/suscripción

De conectar cada sistema con cada otro a publicar una vez y leer muchas

Sin intermediario: conexiones punto a punto



3 × 3 = 9 integraciones que mantener; si un sistema cae, arrastra a otros

Con Kafka: productores y consumidores desacoplados

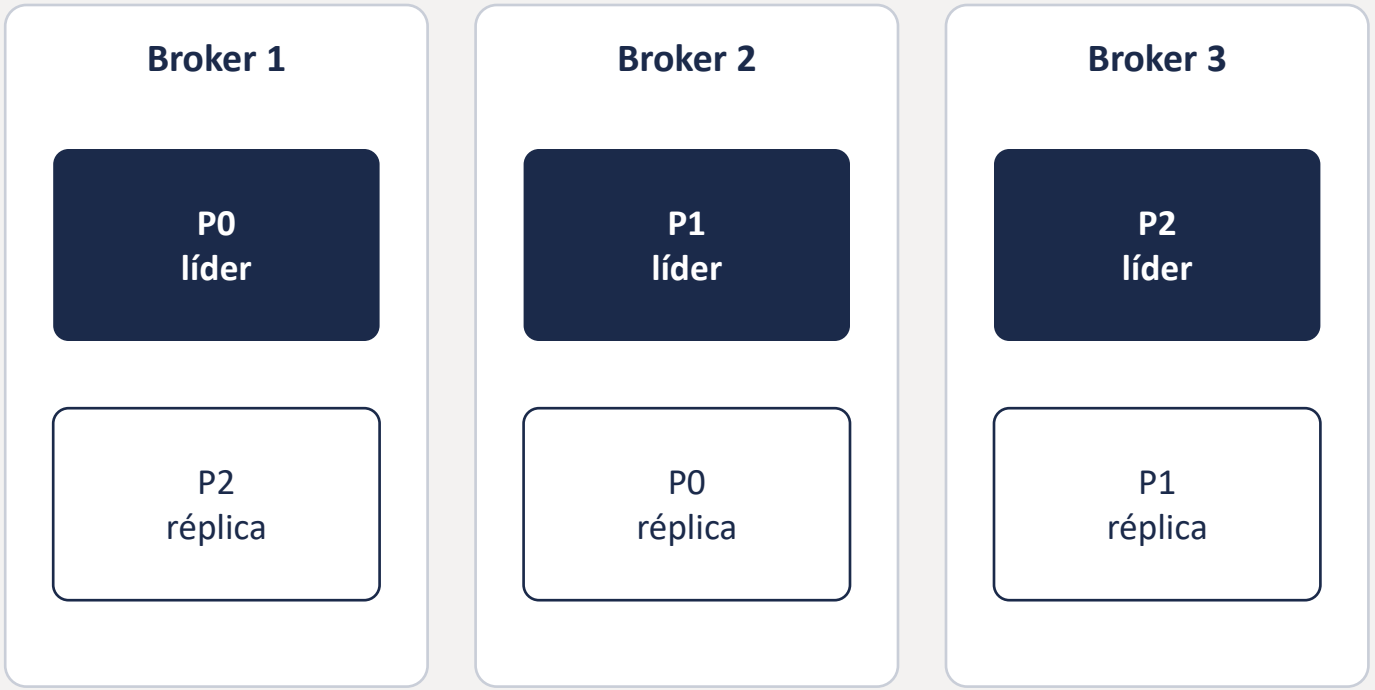


3 + 3 = 6 conexiones; nadie espera a nadie

- **Desacoplamiento:** la app publica en un tema sin saber quién ni cuántos leerán.
- **Cada consumidor a su ritmo:** la tarifa dinámica lee en milisegundos; el archivo histórico puede leer cada hora.
- **Los eventos quedan guardados:** un consumidor nuevo puede leer también lo que ya se publicó.

Brokers, réplicas y líder: qué pasa si un servidor cae

Tema «viajes» con 3 particiones y factor de replicación 2



Líder: atiende escrituras y lecturas

Réplica: copia sincronizada del líder

Partición ≠ réplica. Las particiones reparten el trabajo; las réplicas protegen contra la caída de un servidor.

- **Broker:** cada servidor del clúster de Kafka.
- **Partición:** divide el tema para trabajar en paralelo. Todos los eventos de un viaje van a la misma partición (la clave es el viaje), así llegan en orden.
- **Réplica:** copia de una partición en otro broker; el factor de replicación dice cuántas hay.
- **Si cae el broker 1:** la réplica de P0 en el broker 2 toma el liderazgo. No se pierde lo ya replicado.

En el taller de esta tarde: 1 broker, 2 particiones y réplica 1. Sirve para la mecánica, pero no tolera fallos: en producción se usan al menos 3 brokers.

Garantías de escritura y retención

Cuánto riesgo de pérdida acepta cada evento del viaje y por qué leer no es borrar

acks = 0

Cuándo confirma: nunca; el productor envía y sigue sin esperar respuesta.

Riesgo: Puede perder mensajes sin enterarse.

Uso típico: La ubicación del conductor cada pocos segundos: perder una no cambia la decisión.

acks = 1

Cuándo confirma: cuando el líder escribió el mensaje.

Riesgo: Si el líder cae antes de replicar, el mensaje se pierde.

Uso típico: Registros de la aplicación.

acks = all

Cuándo confirma: cuando todas las réplicas sincronizadas lo tienen.

Riesgo: Más latencia, máxima durabilidad.

Uso típico: El pago del viaje. Valor por defecto del productor desde Kafka 3.0.

Retención: Kafka no borra un mensaje cuando alguien lo lee

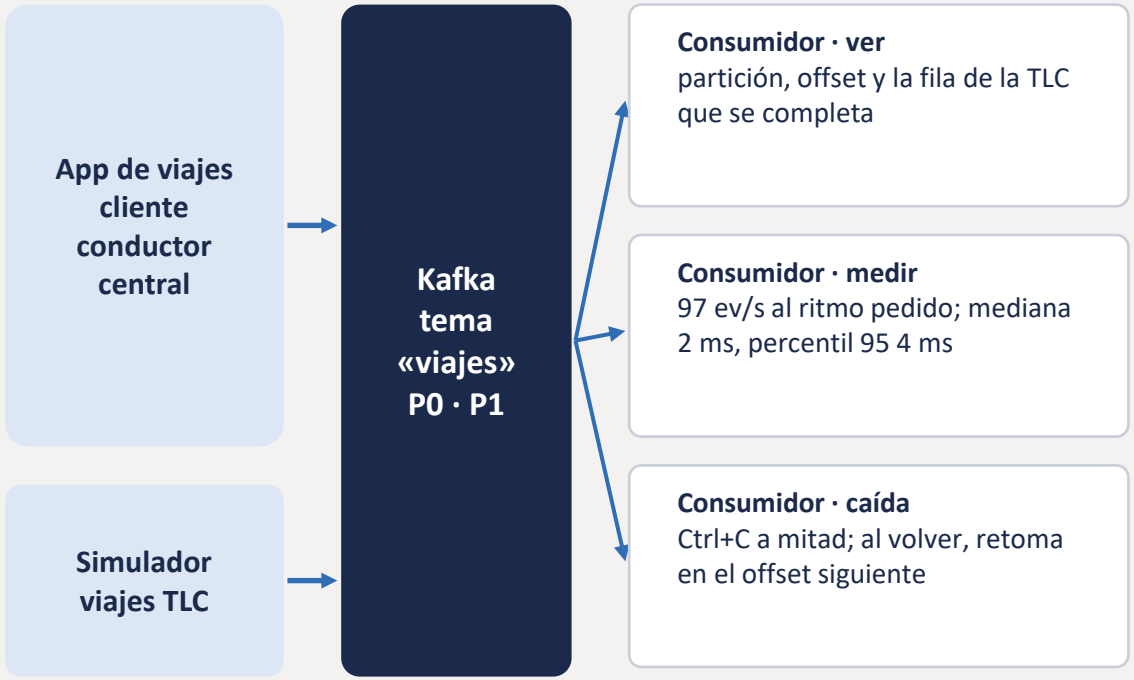
Partición P0: cada mensaje tiene un offset



- Los mensajes se conservan por tiempo o por tamaño (por defecto, 7 días).
- Al volver, el consumidor sigue desde el 4: no pierde lo publicado mientras estaba caído.
- Si vuelve después del plazo de retención, esos mensajes ya no existen.

Taller: los viajes en tiempo real sobre Kafka

1:30–2:50 · Taller de la tarde, Parte A: qué deben observar



- **Un viaje = 6 eventos** (solicitud, asignado, llegada, recogida, fin, pago) con la misma clave: misma partición, en orden.
- **Lag:** lo que le falta por leer a un grupo; si crece sin parar, el consumidor no da abasto.

Escenario: la central se reinicia en plena hora pico

Mientras está caída, los eventos siguen llegando a Kafka y quedan guardados.

Sin offsets confirmados
La central empezaría desde cero o desde el final: repetiría o perdería viajes.

Con grupo de consumidores
Retoma exactamente donde quedó: ni pierde ni repite.

Evidencia a registrar: throughput, latencia, offset de reinicio y lag.

Spark Streaming: ventanas y watermark

2:50–3:45 · Taller de la tarde, Parte B: la espera por zona, mientras ocurre

Ventanas de 15 minutos sobre la hora del viaje

00:00	00:15	00:30	00:45

Cada 10 segundos (un lote) se actualizan las ventanas que cambiaron



Watermark = 30 min antes — Evento más reciente visto

Llega tarde, dentro de 30 min: se agrega a su ventana.
Más atrasado: su ventana ya se cerró y Spark liberó su memoria.

- **Ventana:** agrupa por la hora en que ocurrió el viaje, no por la hora en que llegó el evento.
- **Watermark:** cuánto atraso se acepta; sin él, un flujo que nunca termina guardaría todas las ventanas para siempre.

Escenario: LaGuardia a medianoche (datos reales del 2 de junio)

La espera promedio pasó de 6,9 min (00:00, 162 viajes) a 14,2 min (00:15, 47 viajes).

Señal
Espera que sube en una zona con muchos viajes: faltan conductores.

Decisión
Subir la tarifa o dar incentivos a los conductores cercanos, ahora y no mañana.

El flujo convierte eventos en una señal accionable cada 10 segundos.

Qué cambia en el análisis de datos

La tecnología de procesamiento decide qué preguntas puede responder el analista, con qué datos y a tiempo

	Sin tecnología de procesamiento		Con la evidencia de hoy
Completitud	Excel no abre el mes y de la semana analizó menos de 1 de cada 4 viajes, sin avisar.	→	El 100 % de los 19,9 millones de viajes.
Exactitud	US$ 28,1 millones de tarifa en vez de 132,1 de la semana: un total equivocado que parece correcto.	→	US$ 556,9 millones, idéntico al centavo en Hadoop y Spark.
Oportunidad	El informe por lotes responde al día siguiente.	→	La espera por zona se actualiza cada 10 segundos.
Escala	Un mes pide unos 11 GB en un solo equipo; un año, doce veces más.	→	Más bloques y más tareas: se agregan máquinas, no se cambia el análisis.
Costo	Leer 3,8 GiB de CSV para usar 3 columnas.	→	Parquet: 71 MiB leídos. En la nube se paga por lo que se lee.
Preguntas nuevas	¿Qué pasó el mes pasado?	→	¿Dónde faltan conductores ahora mismo y qué hacemos?

El analista no solo elige el modelo: elige, o debe exigir, la tecnología que hace posible un análisis completo, exacto, oportuno y a un costo razonable.

Cierre del día

Hoy, en GitHub Codespaces

Límite de un PC | HDFS y YARN | MapReduce y combiner | Spark y Parquet | Proyecto Final | Kafka | Offsets y lag | Ventanas y watermark

Próximo encuentro (6 y 7 de noviembre): el mismo caso en la nube

1. **Preparación Previa E2: cuenta de Google Cloud (prueba gratuita o en pareja)**
2. **Clúster de Spark en Google Cloud con los datos en Cloud Storage**
3. **BigQuery y tablero en Looker Studio**
4. **Sustentación del Proyecto Final y del caso empresarial**

> *Apaguen el codespace (F1 → Stop Current Codespace): los datos quedan para el Proyecto Final y no se gastan horas.*